

ME 325 · MECHANICAL ENGINEERING DESIGN PROJECT
MID-TERM PROGRESS

Voice-Actuated Tracking System for Twin Rotor

Voice Track

· *Geometric Control* · *AI Agent Orchestration* · *Human-in-the-Loop Safety*

Group Members:

E/21/338 Riswan

E/21/205 Shathurshiga

E/21/100 Dilshan

Supervisor: Dr. D. H. S. Maithripala

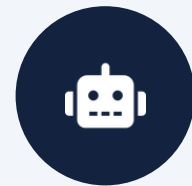
Project Objectives

Develop a modular, voice-actuated control system for the Twin Rotor V1.0 that enables hands-free trajectory tracking — bridging human interaction and aerospace hardware through speech recognition, AI orchestration, and nonlinear control.



Hands-free voice control

Replace manual setpoint entry with natural speech commands, parsed and validated before execution.



AI-driven trajectory generation

A multi-agent orchestrator interprets operator intent and produces precise pitch/yaw trajectories.



Human-in-the-loop safety

No command reaches the rotors without an explicit, simulation-backed operator approval.



Geometric trajectory tracking

Move beyond linear PID to nonlinear SE(3) control for robust large-angle manoeuvres.

Progress

Milestone	Due	Completion	Remarks
Literature review	Week 2	100%	TRMS control, geometric tracking, Whisper/LLM orchestration surveyed
System architecture	Week 3	100%	HiL pipeline defined: App → AI Agent → Simulation → Pi
Mobile app (Flutter)	Week 5	100%	5 screens built: connect, status, voice, HiL approval, telemetry
Basic 3D simulation	Week 5	85%	MeshCat kinematic preview running for pitch/yaw motion
Voice → intent parsing	Week 5	85%	Using Gemini (free tier) in place of Whisper to control cost
App ↔ laptop backend link	Week 6	100%	WebSocket/HTTP connectivity issue
Mobile app ↔ simulation pipeline	Week 6	100%	Voice command from app successfully kick-starts MeshCat simulation on laptop
Hardware integration (TRMS)	Week 9	10%	Connect laptop to Pi and verify handshake protocols

Justification of Achieved Results

Mobile application & pipeline integration — Dilshan

A 5-screen Flutter application is built and fully connected to the laptop backend. Voice commands issued through the app now successfully communicate with the AI agent and kick-start the MeshCat simulation — the complete software pipeline is verified end-to-end.

- ✓ Connect / Status screen — Wi-Fi target selection, live pitch/yaw/PWM readout ✓
- ✓ Voice Command screen — Gemini ASR transcription, parsed-intent chips ✓
- ✓ Trajectory Approval (HiL) screen — 2D preview gate before any hardware write ✓
- ✓ Telemetry screen — fl_chart live plot of pitch vs. target, system log ✓
- ✓ App ↔ laptop WebSocket/HTTP connection resolved and tested ✓

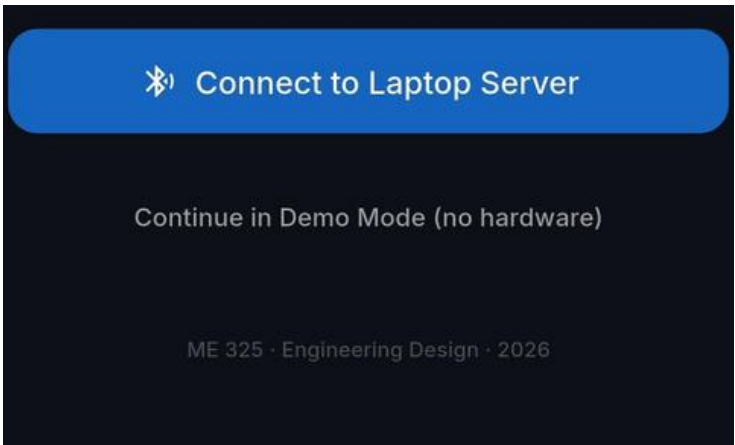
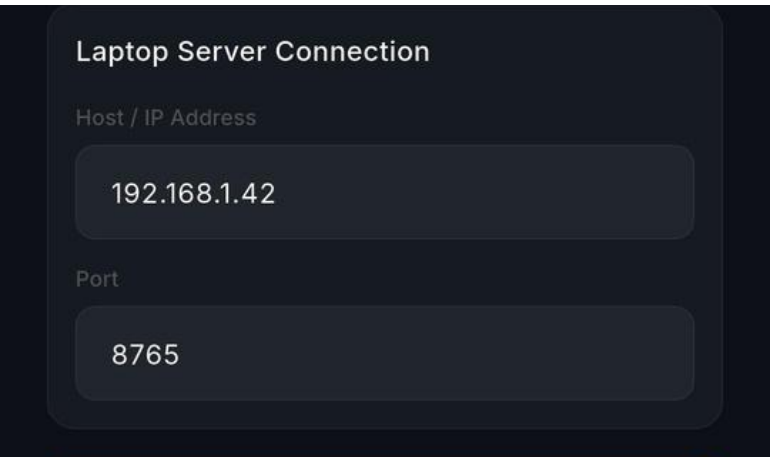
Pipeline status

 **Voice → Gemini → Agent**

 **App ↔ Backend: Connected**

 **HiL Approval Gate: Active**

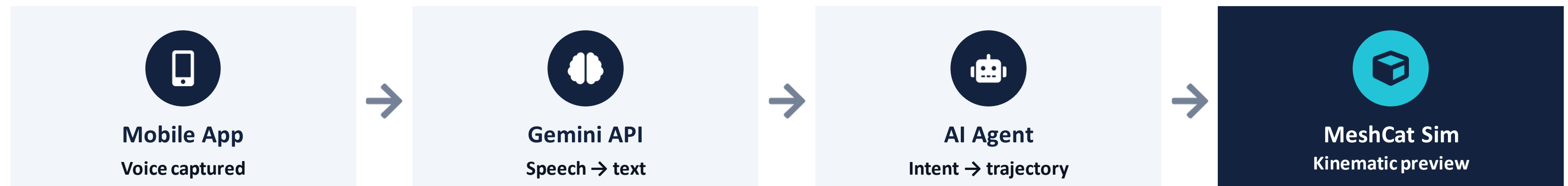
 **Simulation: Kick-starts ✓**



Justification of Achieved Results

Full pipeline operational: simulation connected to app — Shathurshiga, Riswan

The complete software pipeline is now operational end-to-end: a voice command on the mobile app is transcribed by Gemini, parsed by the AI agent, and automatically kick-starts the MeshCat 3D simulation on the laptop — with Human-in-the-Loop approval gating before any hardware command is issued.



Engineering trade-off: ASR provider

OpenAI Whisper requires paid API credits.

Switched to the Gemini API (free tier) for speech-to-intent during development to keep iteration cost-free.

Will re-benchmark against Whisper once accuracy is validated on lab audio — keeping the ASR layer swappable was a deliberate design choice.

Validated this iteration

- Full voice-to-simulation pipeline verified end-to-end ✓
- Mobile app ↔ laptop WebSocket/HTTP connection resolved ✓
- MeshCat renders pitch/yaw motion live; HiL approval gate enforced ✓

Justification of Achieved Results

What's resolved — and what's next



Software pipeline fully resolved — all components connected

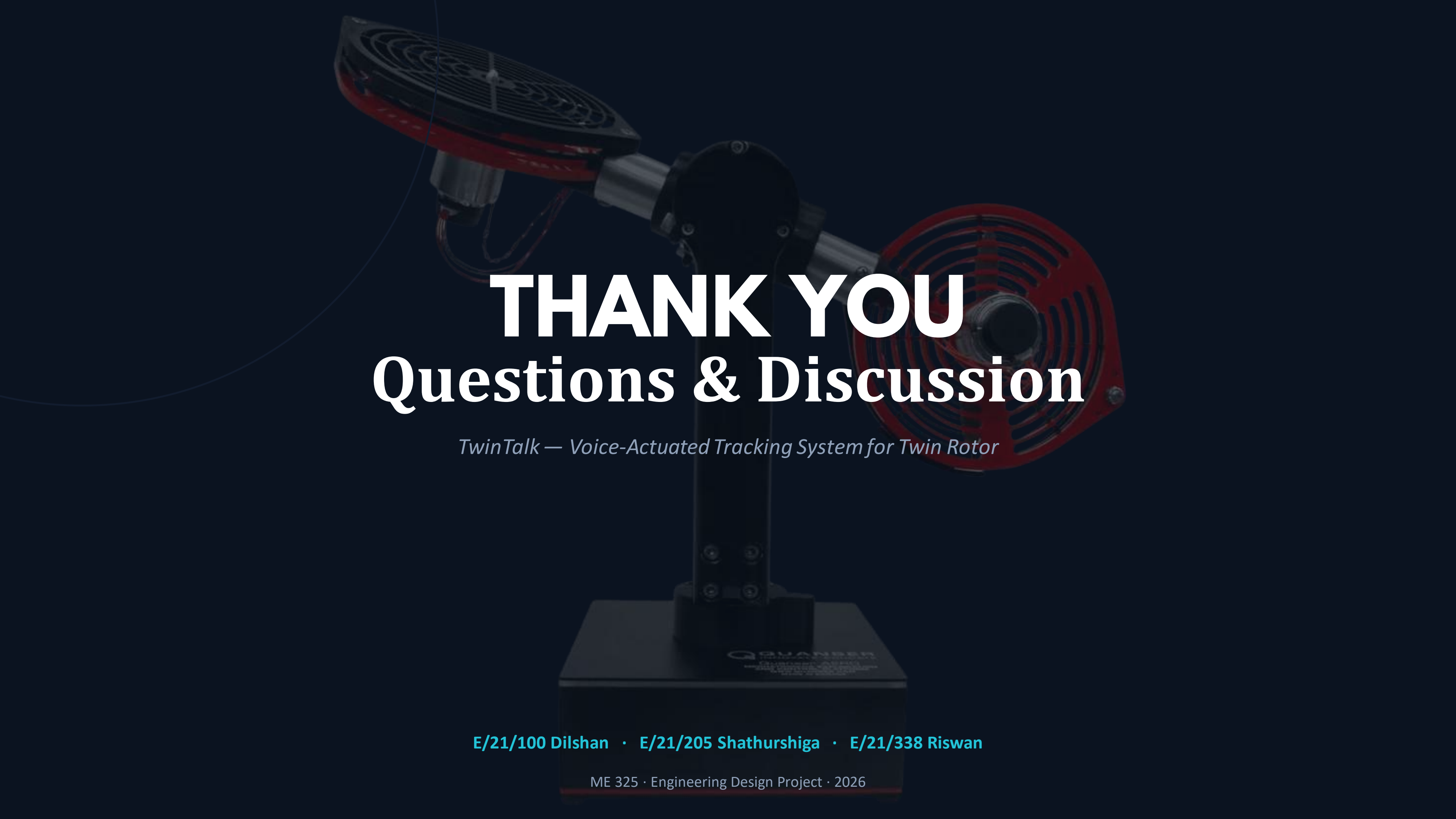
Mobile app ↔ FastAPI backend ↔ LangGraph AI agent ↔ MeshCat simulation: the full chain is operational. Voice command on phone kick-starts simulation on laptop with HiL approval enforced.

Remaining work

- **Connect to physical Twin Rotor (TRMS)**
Wire Raspberry Pi PWM output to main and tail rotors; confirm torque response matches simulation
- **PID baseline control on hardware**
Bring up linearized PID closed-loop first; confirm pitch/yaw hold before upgrading
- **Benchmark Gemini vs. Whisper on lab audio**
Decide final ASR provider once both are tested with rotor noise in the background
- **Upgrade to geometric SE(3) control**
Replace PID with nonlinear controller once hardware baseline is verified stable
- **Final report & demo video**
Document all results; record end-to-end voice-to-rotor demonstration

Hardware integration plan

- Step 1 — Pi server**
Deploy WebSocket server on Raspberry Pi; smoke-test PWM output with oscilloscope
- Step 2 — Safety test**
HiL gate forces simulation preview before any rotor command; E-stop enforced at app level
- Step 3 — PID bring-up**
Linearized PID tuned in MATLAB/Simulink; port to Python on Pi and close the loop
- Step 4 — Geometric upgrade**
SE(3) controller replaces PID for large-angle robustness; validate against simulation results



THANK YOU

Questions & Discussion

TwinTalk — Voice-Actuated Tracking System for Twin Rotor

E/21/100 Dilshan · E/21/205 Shathurshiga · E/21/338 Riswan

ME 325 · Engineering Design Project · 2026